



UNIVERSIDADE D
COIMBRA

UNIVERSIDADE DE COIMBRA
FACULDADE DE CIÊNCIAS E TECNOLOGIAS
DEPARTAMENTO DE ENGENHARIA INFORMÁTICA

Privacidade Através de Encriptação Homomórfica

Security and Privacy

Authors:

Afonso RODRIGUES – 2020242400

David FERREIRA – 2020248052

João FERNANDES — 2021219036

Professor:

Naghmeh IVAKI

19 de dezembro de 2024

Conteúdo

1	Introdução	1
2	Metodologia	1
2.1	Implementação do <i>Data Holder</i>	2
2.2	Implementação do <i>Data Analyzer</i>	3
3	Análise de Resultados	4
4	Discussão	5
5	Conclusões	5

1 Introdução

Num mundo cada vez mais digital, a privacidade dos dados tornou-se uma preocupação fundamental. Dados sensíveis, como informações pessoais, financeiras e de saúde, são frequentemente transmitidos e processados através de redes, o que os torna vulneráveis a violações. Os métodos tradicionais de encriptação protegem os dados durante o armazenamento e a transmissão, porém **falham quando são efetuados cálculos nos dados encriptados**. Esta lacuna exige uma procura de técnicas como a **Encriptação Homomórfica (*Homomorphic Encryption*, HE)**.

A HE é uma forma de encriptação que permite que os cálculos sejam efetuados diretamente nos dados encriptados sem necessidade de desencriptação. Os resultados dos tais cálculos permanecem encriptados e só podem ser desencriptados pelo proprietário dos dados.

O principal objetivo deste projeto é demonstrar a implementação e aplicação da HE utilizando uma simulação como setup que envolve um *Data Holder* e um *Data Analyzer*. Especificamente, os objetivos incluem:

- Implementar um *Data Holder* para encriptar o conjunto de dados.
- Efetuar cálculos nos dados encriptados utilizando o *Data Analyzer*.
- Desencriptar os resultados utilizando novamente o *Data Holder* e interpretar os resultados para validar a correção do cálculo.

Utilizamos bibliotecas e estruturas para encriptação homomórfica, como o PyCrypto ou ferramentas equivalentes. Os dados são encriptados utilizando técnicas de HE, os cálculos são efetuados em *dataframes* encriptados e os resultados desencriptados são analisados graficamente.

2 Metodologia

Esta experiência envolve duas entidades principais:

- **Data Holder:** Encripta o conjunto de dados antes de o partilhar.
- **Data Analyzer:** Efetua cálculos em dados encriptados.

As ferramentas utilizadas para a implementação incluem bibliotecas na linguagem Python, tais como *TenSEAL* para encriptação homomórfica, *pandas* para manipulação de dados e armazenamos os dados num ficheiro *.json*.

2.1 Implementação do *Data Holder*

O *Data Holder* é responsável pela encriptação do conjunto de dados e pela desencriptação dos resultados recebidos do *Data Analyzer*. Consiste nas seguintes funções:

- `encrypt_earnings(dataframe, file_name):`

```
def encrypt_earnings(dataframe, file_name):
    # Criar o contexto para CKKS
    context = ts.context(
        ts.SCHEME_TYPE.CKKS, poly_modulus_degree=8192, coeff_mod_bit_sizes=[60, 40, 40, 60]
    )
    context.generate_galois_keys()
    context.global_scale = 2**40

    # Criptografar a coluna Earnings
    encrypted_earnings = [ts.ckks_vector(context, [earning]) for earning in dataframe["Earnings"]]

    # Preparar dados para salvar
    encrypted_data = {
        "context": base64.b64encode(context.serialize(save_secret_key=True)).decode('utf-8'), # Usando base64
        "encrypted_earnings": [e.serialize().decode('latin1') for e in encrypted_earnings],
        "years": dataframe["Year"].tolist(),
        "names": dataframe["Name"].tolist(),
    }

    # Salvar os dados criptografados
    with open(file_name, "w") as f:
        json.dump(encrypted_data, f)

    print(f"Earnings criptografados salvos em {file_name}.")
```

- Esta função encripta a coluna dos ganhos do conjunto de dados de entrada utilizando o esquema CKKS da biblioteca *TenSEAL*. Um contexto é gerado com parâmetros específicos para garantir a segurança.
- Os dados encriptados, incluindo o contexto, os anos e os nomes, são armazenados num ficheiro JSON para posterior computação.
- O contexto é *serializado* utilizando a codificação base64 para preservar as chaves de encriptação de forma segura.

- `decrypt_results(results_file):`

```
def decrypt_results(results_file):
    with open(results_file, "r") as f:
        results = json.load(f)

    # Restaurar o contexto com chave secreta a partir de base64
    context = ts.context_from(base64.b64decode(results["context"]))

    # Verificar se a chave secreta está presente
    print("Chave secreta presente após restaurar:", context.has_secret_key())

    # Descriptografar soma e média
    decrypted_sum = ts.ckks_vector_from(context, results["encrypted_sum"].encode('latin1')).decrypt()
    decrypted_mean = ts.ckks_vector_from(context, results["encrypted_mean"].encode('latin1')).decrypt()
    encrypted_variancia = ts.ckks_vector_from(context, results["encrypted_variancia"].encode('latin1')).decrypt()

    # Arredondar e exibir os resultados (caso sejam listas)
    rounded_sum = [round(value, 2) for value in decrypted_sum] # Arredondar cada valor da soma
    rounded_mean = [round(value, 2) for value in decrypted_mean] # Arredondar cada valor da média

    # Mostrar os resultados descriptografados
    #print(f"Soma descriptografada: {rounded_sum}")
    #print(f"Média descriptografada: {rounded_mean}")
    return rounded_sum, rounded_mean, encrypted_variancia
```

- Esta função desencripta os resultados recebidos do *Data Analyzer*, restaura o contexto de encriptação e desencripta a soma, a média e a variância dos ganhos.
- Os valores desencriptados são arredondados para facilitar a leitura e devolvidos.

2.2 Implementação do *Data Analyzer*

O *Data Analyzer* é responsável pela realização dos cálculos nos *dataframes* encriptados partilhados pelo *Data Holder*. Efetua a seguinte operação:

- `analyse_encrypted_earnings(input_file, output_file):`

```
def analyse_encrypted_earnings(input_file, output_file):
    # Carregar o arquivo criptografado
    try:
        with open(input_file, "r") as f:
            encrypted_data = json.load(f)
        except Exception as e:
            print(f"Erro ao carregar o arquivo {input_file}: {e}")
            return

    # Verificar se os dados foram criptografados corretamente
    if "context" not in encrypted_data or "encrypted_earnings" not in encrypted_data:
        print("Todos os arquivos estão incompletos ou mal formatados.")
        return

    # Restaurar o contexto com chave secreta a partir de base64
    context = base64.b64decode(encrypted_data["context"])

    # Verificar se a chave secreta está presente
    print("Chave secreta presente após restaurar:", context.has_secret_key())

    # Restaurar os dados criptografados
    encrypted_earnings = []
    for i, e in enumerate(encrypted_data["encrypted_earnings"]):
        encrypted_earnings.append(e)

    # Serializar os dados
    serialized_earnings = [e.serialize() for e in encrypted_earnings]

    # Calcular estatísticas
    encrypted_sum = sum(serialized_earnings)
    encrypted_mean = encrypted_sum / len(serialized_earnings)
    encrypted_sum_square = sum([e.serialize() ** 2 for e in encrypted_earnings])
    encrypted_variance = (encrypted_sum_square / len(serialized_earnings)) - (encrypted_mean ** 2)

    # Preparar resultados para salvar
    results = {
        "encrypted_sum": encrypted_sum.serialize().decode('latin1'),
        "encrypted_mean": encrypted_mean.serialize().decode('latin1'),
        "encrypted_variance": encrypted_variance.serialize().decode('latin1'),
        "context": encrypted_data["context"], # A chave secreta
    }

    # Salvar os resultados criptografados
    with open(output_file, "w") as f:
        json.dump(results, f)

    print(f"Resultados criptografados salvos em {output_file}.")
```

- A função carrega os dados encriptados, restaura o contexto de encriptação e executa operações como soma, média e variância na coluna de ganhos encriptados.
- Os resultados dessas operações são *serializados* e salvos num ficheiro de saída.

O desempenho da implementação da HE foi avaliado com base em:

- **Precisão computacional:** Assegurar que os resultados descriptados correspondem aos resultados esperados.
- **Eficiência:** Medir o tempo de execução das etapas de encriptação, computação e descriptação.
- **Segurança:** Manter a privacidade durante a análise de dados.

Algumas limitações da metodologia escolhida:

- **Sobrecarga computacional:** Os métodos de HE podem ser mais lentos do que a encriptação tradicional.
- **Complexidade de implementação:** A implementação da HE requer bibliotecas especializadas e é computacionalmente intensiva.

3 Análise de Resultados

Análise das métricas de desempenho (velocidade, eficiência, segurança):

- **Tempo de encriptação:** O processo de encriptação demorou 6 minutos e 20 segundos.
- **Tempo de computação:** Os cálculos sobre os dados encriptados foram efetuados em 6 minutos.
- **Tempo de desencriptação:** A desencriptação dos resultados demorou 3 segundos.7

Com isto podemos tirar a conclusão de que o processo de encriptação é muito mais demorado que o processo de decriptação, bem como os cálculos em dados encriptados.

Foi também realizada uma análise relativa à **média, soma e variância** antes e depois dos dados encriptados.

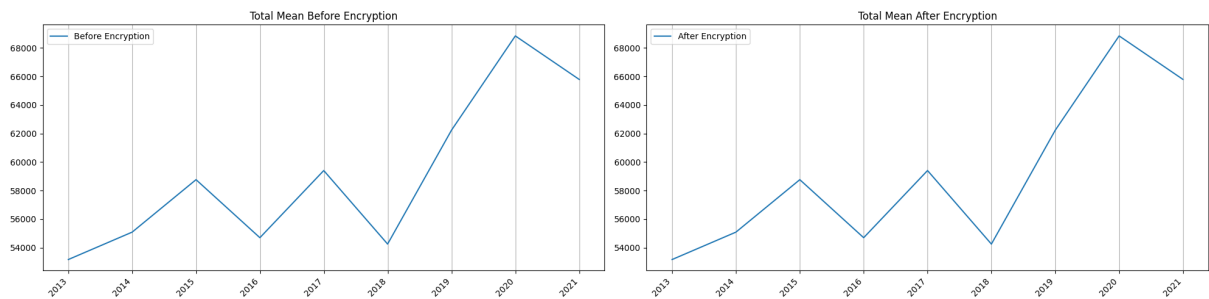


Figura 1. Médias antes e depois da encriptação

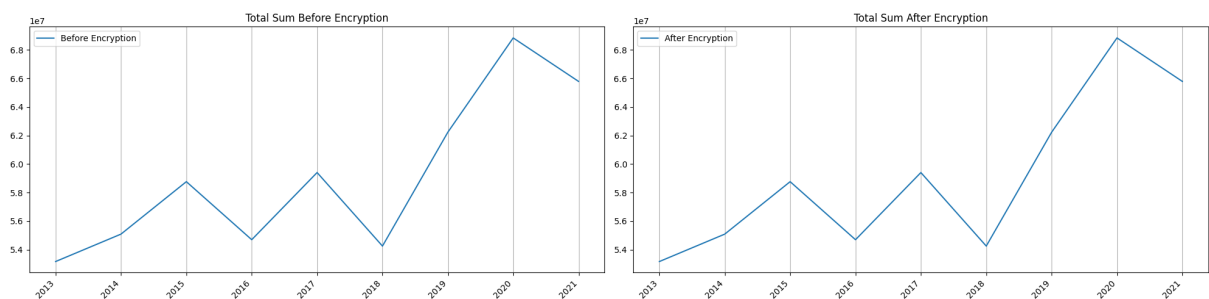


Figura 2. Somas antes e depois da encriptação

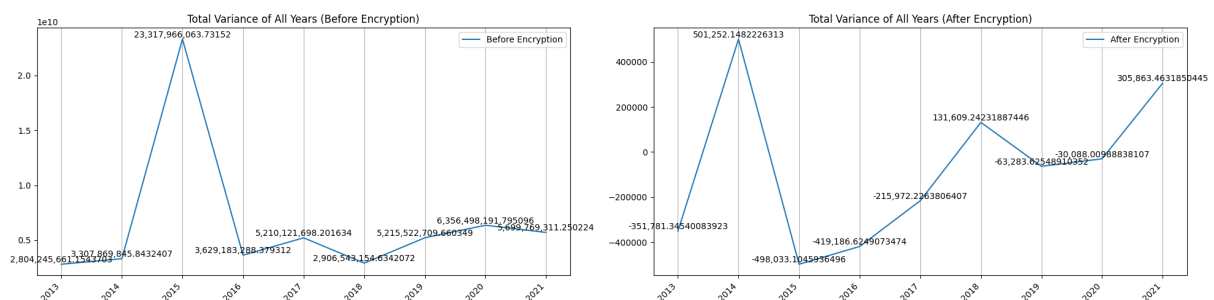


Figura 3. Variâncias antes e depois da encriptação

Como podemos analisar pelas imagens, os valores tanto da média como da soma batem certo, sendo estes exatamente os mesmos tanto antes como depois da encriptação dos dados. Porém, quanto à variância notamos uma grande diferença relativa não só à escala mas também quanto aos dados.

Uma vez que a soma dos valores por ano mantém-se a mesma, é seguro assumir que valores não foram removidos, e uma vez que a média também se mantém a mesma é seguro assumir que se houve alteração dos dados, não seria impactante. Porém, suspeitamos que tenha a ver com o facto de ao deciptar os dados encriptados, as imprecisões eixstentes são amplificadas. Uma outra opção pode ser que a encriptação dos dados foca-se simplesmente me manter uma média dos dados,

4 Discussão

Esta experiência demonstrou que a encriptação homomórfica permite efetuar cálculos de forma segura em dados encriptados, indo ao encontro dos objetivos do projeto. Os resultados provaram que a HE consegue manter a privacidade dos dados sem comprometer alguma exatidão analítica.

A implementação bem-sucedida da HE realça o seu potencial para aplicações do mundo real em computações *privacy-preserving*, como a análise baseada na cloud e a partilha segura de dados.

Pontos fortes da Homomorphic Encryption utilizada:

- Garante que os dados permanecem privados durante a análise.
- Resultados computacionais corretos sem descodificação dos dados.

Pontos fracos da Homomorphic Encryption utilizada:

- Aumento da sobrecarga computacional.
- Complexidade de implementação e configuração.

5 Conclusões

Como referido, a HE é um avanço significativo na privacidade dos dados, permitindo cálculos seguros sem expor os dados em bruto. Aborda os principais desafios em cenários que envolvem a análise de dados sensíveis. Este projeto demonstrou a utilização bem-sucedida da HE para a análise segura de dados. Embora a sobrecarga computacional continue a ser uma limitação, a abordagem garante uma privacidade de dados robusta e resultados analíticos precisos, tornando-a relevante para aplicações em ambientes seguros para partilha de dados.